An 8-bit Breadboard Computer from Scratch

Sabiha Mahabub Shammi
ID: 2108002
Dept. of ETE, CUET

Bonoshree Talukdar Tondra
ID: 2108009
Dept. of ETE, CUET

Tasnia Afrin
ID: 2108050
Dept. of ETE, CUET

***Abstract*—This paper presents the design and implementation of an 8-bit breadboard computer built entirely from discrete logic components. The system is inspired by the Ben Eater 8-bit computer architecture and is developed to demonstrate fundamental concepts of computer organization at the hardware level. The proposed computer consists of a clock generator, program counter, register, arithmetic and logic unit (ALU), control unit, memory module, and a shared 8-bit data bus. A microcode-based control scheme is used to generate the required control signals for instruction execution. The processor supports a basic instruction set including data transfer, arithmetic operations, conditional and unconditional branching, and output display. The complete system is implemented on solderless breadboards, experimental testing confirms that the computer can correctly fetch, decode, and execute programs stored in memory, producing accurate results on the output module. This project provides an effective platform for understanding low-level CPU operation, digital logic design, and the relationship between hardware and machine-level instructions.**

I. Introduction

Understanding how a computer executes instructions at the hardware level is fundamental to the study of computer and digital system design. Although modern processors are highly complex and integrated, their operation is still based on simple principles such as data movement through registers, arithmetic operations, control signals, and clock-driven state transitions. These internal processes are often hidden behind software abstractions, making it difficult to visualize how machine-level instructions are actually executed by physical hardware. Therefore, constructing a processor from basic digital logic components provides an effective way to bridge theory and practical understanding.

An 8-bit computer represents a minimal yet complete processing system capable of performing arithmetic, logic, memory access, and program control. Despite its simplicity, such a system contains all essential elements of a modern CPU, including registers, an arithmetic and logic unit (ALU), a program counter, memory, and a control unit. Implementing these components on breadboards using discrete logic integrated circuits allows the behavior of each block to be directly observed and tested. This hands-on approach is particularly valuable for learning how data buses, control signals, and clock synchronization govern the operation of a digital processor.

The objective of this work is to design, implement, and experimentally validate an 8-bit breadboard computer capable of executing a basic instruction set including data transfer and arithmetic operations. The completed system demonstrates the full instruction cycle—from fetching and decoding to execution—using only discrete digital components. Through this implementation, the project provides a clear and practical demonstration of fundamental computer architecture and digital system concepts.

II. System Overview

A. Clock Module

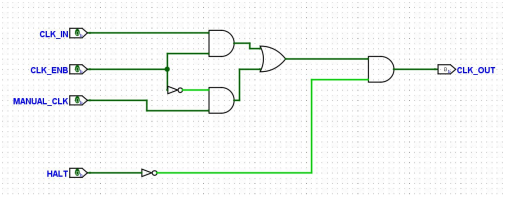


Fig. 1. Clock module of the 8-bit breadboard computer

The clock module provides the timing reference for the entire computer. It generates periodic clock pulses that drive the state transitions of all registers and control logic. Both manual single-step and automatic continuous clock modes are supported, allowing controlled debugging as well as normal program execution. The clock signal ensures that data is placed on and removed from the shared bus in a synchronized manner, preventing conflicts between modules.

B. Register

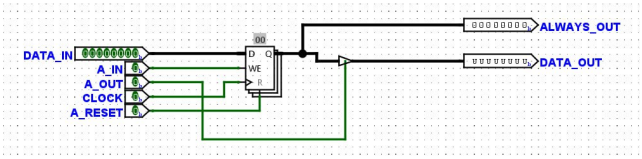


Fig. 2. Register A

The register bank consists of several 8-bit registers that temporarily store data during instruction execution. These include the accumulator (A register), a secondary operand register (B register), an instruction register, and internal registers used by the control unit.

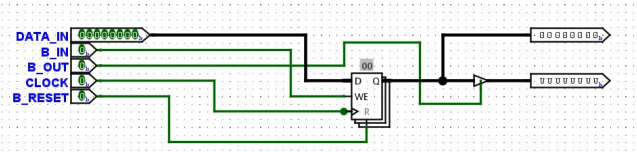


Fig. 3. Register B

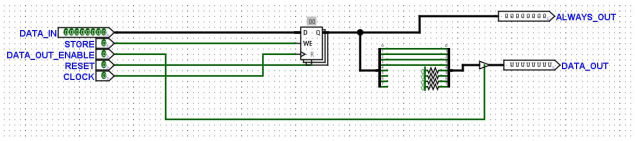


Fig. 4. Instruction Register

Each register can place its stored value onto the bus or load new data from the bus under the control of enable and load signals. This modular register structure allows data to be transferred efficiently between memory, the ALU, and the output unit.

C. Arithmetic and Logic Unit (ALU)

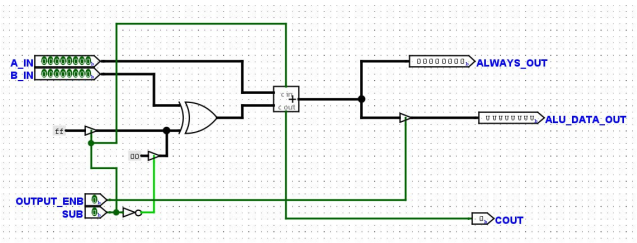


Fig. 5. Arithmetic and Logic Unit

The ALU performs arithmetic and logical operations on 8-bit data using inputs from the A and B registers. It generates an 8-bit result for the system bus and produces status flags, such as carry and zero, to enable conditional branching under control-unit signals.

D. Random Access Memory (RAM)

The RAM module stores program instructions and data, supplying instructions during the fetch phase and supporting data read/write operations during execution via the shared bus. Its limited size is adequate for demonstrating basic program execution.

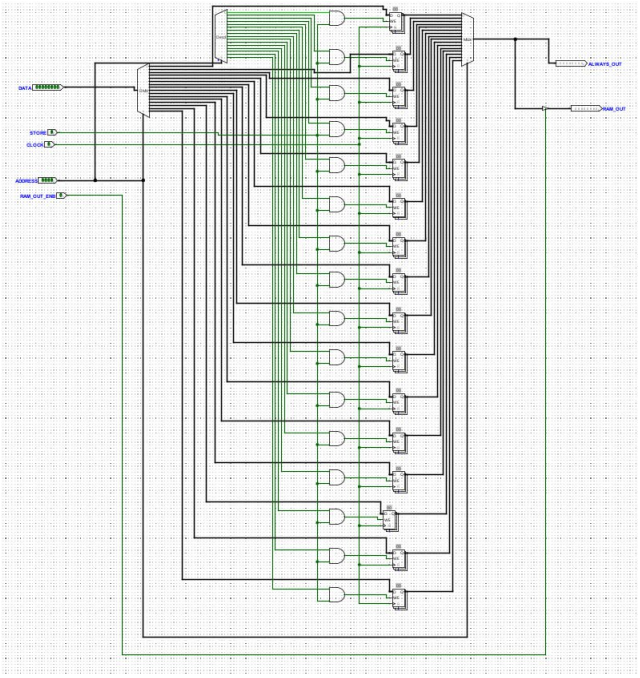


Fig. 6. Random Access Memory

E. Program Counter

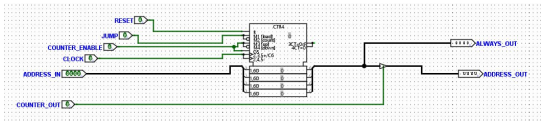


Fig. 7. Program Counter

The program counter stores the address of the next instruction and increments automatically after each fetch. It can be updated during jump or branch instructions to control program flow, enabling sequential instruction execution.

F. Output Register

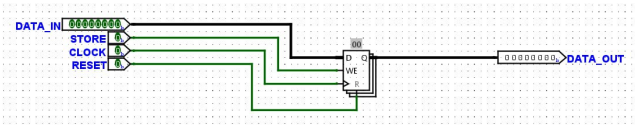


Fig. 8. Random Access Memory

The output register stores bus data and drives external displays, allowing computation results to be directly observed and providing a simple interface between the CPU and the user. It updates only when an output instruction is executed, ensuring controlled data presentation. This module helps verify system operation and simplifies debugging during program execution.

III. Hardware Design

The hardware design of the 8-bit breadboard computer was based on a modular architecture, where each functional unit was implemented as an independent circuit block. The system was constructed using standard TTL 74LS-series integrated circuits and interconnected through a shared 8-bit data bus. This modular approach simplified debugging, testing, and future expansion.

A centralized data bus was employed to enable controlled data transfer between registers, memory, and the arithmetic logic unit (ALU). Tri-state bus transceivers were used to ensure that only one module drove the bus at any given time, thereby preventing bus contention. All modules operated synchronously under a common clock signal.

Control signals were generated through combinational and sequential logic circuits to coordinate register loading, bus enabling, memory access, and ALU operations. Each module was designed to respond only when its corresponding control signals were asserted, ensuring orderly execution of instructions.

The hardware design emphasized clarity of signal flow, proper timing synchronization, and physical feasibility on breadboards. Individual modules were first validated independently before full system integration to reduce complexity during debugging.

A. Clock Circuit

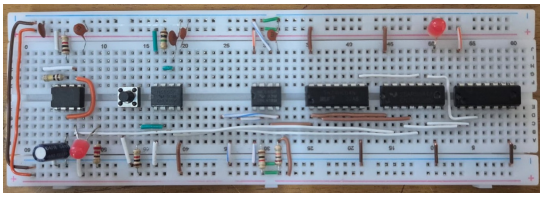


Fig. 9. Clock Circuit

B. Register Implementation

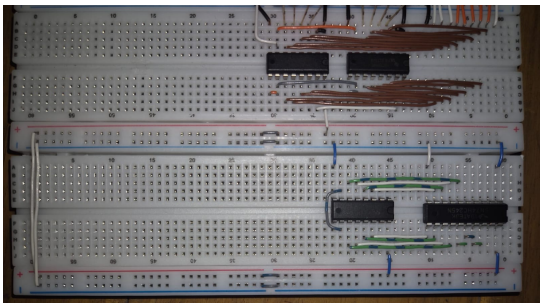


Fig. 10. Registers

C. ALU Circuit

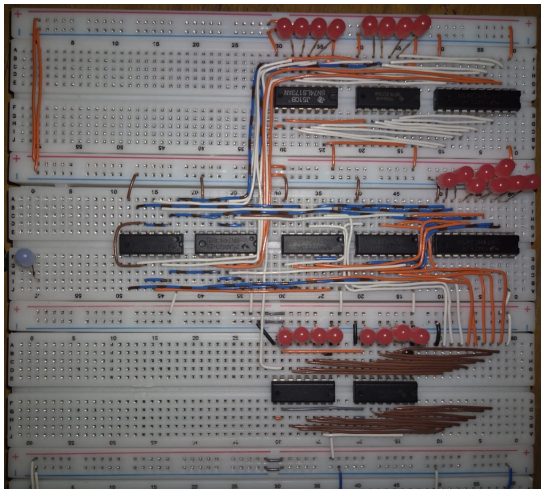


Fig. 11. ALU Circuit

D. Program Counter Circuit

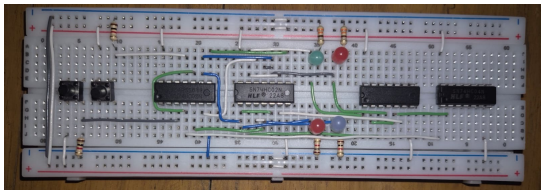


Fig. 12. Program Counter

IV. Implementation and Verification

A. Simulation and Verification

Prior to physical assembly, the CPU design was simulated using Logisim, a digital logic simulator. Simulation allowed verification of correct instruction fetch, decode, and execution cycles. Timing diagrams confirmed proper synchronization of the clock, registers, ALU, memory, and control signals.

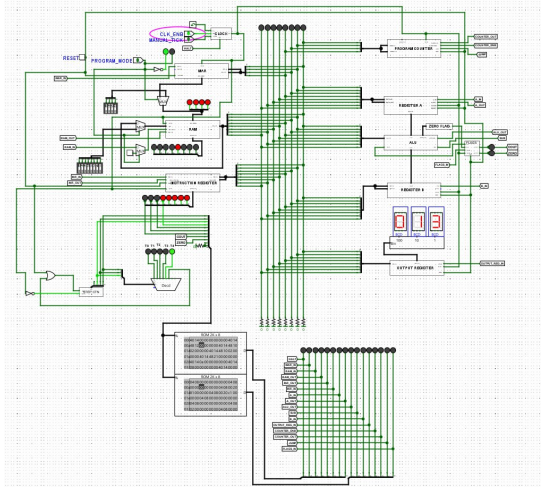


Fig. 13. 8-bit Computer

B. Hardware Assembly

The system modules were assembled on standard breadboards and interconnected via a common 8-bit data bus. Tri-state buffers were employed to prevent bus contention by allowing only one module to drive the bus at a time. Power distribution and signal routing were carefully managed to maintain signal integrity and prevent glitches. Each module was tested incrementally.

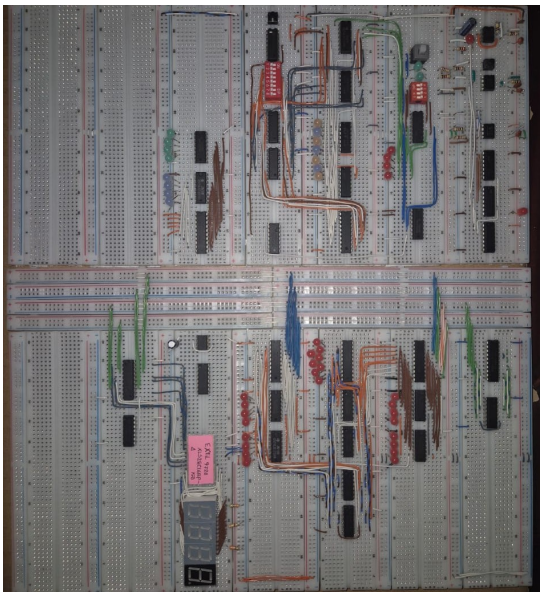


Fig. 14. 8 bit Architecture

- **Clock:** Verified pulse frequency and single-step functionality.
- **Registers:** Verified correct load and output-enable behavior.
- **ALU:** Verified addition, subtraction, and logic operations with test inputs.
- **Program Counter and Memory:** Verified sequential instruction addressing and read/write operations.
- **Output Register:** Verified accurate display of data on LEDs.

Each module passed its individual functional test before integration, minimizing debugging complexity.

V. Limitations and Cost Analysis

The primary limitation of this project was the unavailability of certain critical integrated circuits required for the complete implementation of the 8-bit computer. Specifically, the , a 64-bit RAM IC and the 28C16 EEPROM used for control microcode could not be obtained within the project timeframe. As a result, the full memory and control unit functionality could not be implemented exactly as intended in the original architecture.To address this constraint, alternative ICs and placeholder configurations were used to represent the missing components for architectural understanding and breadboard layout purposes.

TABLE I
Summary of Components and Cost

Component Category	Approx. Cost (BDT)
Breadboards and wiring materials	2,100
Passive components	500
TTL logic ICs	2,200
Memory and control ICs	700
Switches and clock components	400
LEDs and display units	500
Total Cost	**6,500**

Individual subsystems, including the clock module, registers, SR latch, and JK flip-flop circuits, were independently powered and tested and were observed to function correctly. However, when all modules were interconnected to form the complete system, stable operation could not be achieved due to the absence of the required memory and control ICs, leading to incomplete signal coordination across the system.

Despite these limitations, the complete architectural design of an 8-bit computer was successfully developed and implemented on breadboards. The project effectively demonstrates the organization, interconnection, and operation of fundamental CPU building blocks, providing valuable insight into hardware-level computer architecture.

Acknowledgment

Sabiha Mahabub Shammi contributed to the design and hardware implementation of the clock module, register circuits and ALU Design.
Bonoshree Talukdar Tondra was responsible for the simulation verification and memory interfacing.
Tasnia Afrin coordinated the overall system integration, testing and report preparation.
We would like to express our sincere gratitude to our course teacher and faculty members for their guidance and support throughout this project. Also thanks to our seniors, who helped us to test the simulation. The laboratory facilities and technical assistance provided by the department are also gratefully acknowledged.

References

[1] B. Eater, “Build an 8-bit computer,” YouTube. [Online]. Available: https://eater.net/8bit. [Accessed: Jan. 17, 2026].
[2] Texas Instruments, “74LS Series TTL Logic Data Sheet,” Texas Instruments, 2010. [Online]. Available: https://www.ti.com/lit/ds/symlink/sn74ls00.pdf. [Accessed: Jan. 17, 2026].
[3] C. R. Curts, “Logisim: A Graphical Tool for Logic Circuit Simulation,” [Online]. Available: http://www.cburch.com/logisim/. [Accessed: Jan. 17, 2026].
[4] H. M. Deitel and P. J. Deitel, *Computer Architecture*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2012.
[5] M. Morris Mano and Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 6th ed. Pearson, 2017.
The TTL logic used is based on standard 74LS-series ICs [2].